

Conectar la lógica de la aplicación y las interfaces que hemos construido con una base de datos persistente usando EF Core es un paso fundamental.

Material de Estudio: Acceso a Datos con Entity Framework Core

Profesor: Jorge Izaguirre

Materia: Programacion .NET

Curso: 6to Año - TÉCNICO EN PROGRAMACIÓN

Parte 1: El Puente entre Mundos - ORM, EF Core y Code-First

(1.1) El Desafío: Objetos vs. Tablas (Impedance Mismatch)

Hemos aprendido a modelar nuestra aplicación usando Programación Orientada a Objetos (Clases, Herencia, etc.) en C# y a estructurar datos en Bases de Datos Relacionales (Tablas, Filas, Columnas, Relaciones con PK/FK) usando SQL. Sin embargo, estos dos mundos (orientado a objetos y relacional) no encajan perfectamente:

- Los objetos pueden tener relaciones complejas (colecciones de otros objetos, herencia).
- Las bases de datos relacionales usan tablas planas y claves foráneas para representar relaciones.
- Necesitamos una forma de "traducir" entre nuestros objetos C# y las tablas/filas de la base de datos.

La forma tradicional (usando ADO.NET) implica escribir código SQL como strings dentro de nuestro C#, ejecutar esos comandos, y luego mapear manualmente los resultados de la base de datos (filas y columnas) a las propiedades de nuestros objetos C#. Esto es:

- **Tedioso y Repetitivo:** Mucho código "boilerplate" (caldera) para operaciones CRUD básicas.
- **Propenso a Errores:** Errores de tipeo en los strings SQL (no detectados por el compilador), vulnerabilidades de *SQL Injection* si no se parametrizan correctamente las consultas.
- **Difícil de Mantener:** Cambios en la base de datos requieren buscar y modificar todos los strings SQL relacionados en el código C#.
- **Acoplado al Motor de BD:** El SQL escrito puede ser específico para SQL Server y no funcionar directamente en PostgreSQL o MySQL.

(1.2) ORM: El Traductor Automático

Para solucionar estos problemas, surgieron los **ORM (Object-Relational Mappers)**.

- *¿Qué es un ORM?* Es una técnica y un tipo de librería de software que crea un "puente" virtual entre tu base de datos relacional y tu lenguaje de programación orientado a objetos. Automatiza la traducción entre objetos y tablas.
- *¿Cómo funciona (conceptual)?* Defines tus clases en C# (tus Modelos/Entidades) y configuras el ORM para que entienda cómo esas clases se mapean a las tablas de tu base de datos. Luego, en lugar de escribir SQL, interactúas con tus objetos C# y el ORM se encarga de generar y ejecutar el SQL necesario por debajo.
- *Beneficios Clave:*
 - **Mayor Productividad:** Reduce drásticamente la cantidad de código de acceso a datos que necesitas escribir y mantener. Te concentras más en la lógica de negocio.

- **Abstracción:** Trabajas con objetos C# y consultas LINQ, lo que es más natural en un entorno .NET. El ORM maneja los detalles del SQL.
- **Seguridad de Tipos:** Las consultas (usando LINQ) y el acceso a propiedades se verifican en tiempo de compilación, atrapando errores antes.
- **Mantenibilidad:** Los cambios en el modelo se manejan de forma más centralizada.
- **Independencia (Parcial) del Motor de BD:** Usando el mismo código C#/LINQ, podrías cambiar de SQL Server a otro motor soportado por el ORM con cambios relativamente menores (principalmente en la configuración).

(1.3) Entity Framework Core (EF Core): El ORM de .NET

Entity Framework Core (EF Core) es el ORM oficial de Microsoft para .NET. Es moderno, ligero, extensible, open-source y multiplataforma. Es la evolución de Entity Framework 6 (que era solo para .NET Framework en Windows).

- **Enfoques de Desarrollo con EF Core:**
 - *Database-First:* Si ya tienes una base de datos existente, EF Core puede inspeccionarla y generar automáticamente las clases C# (Entidades y `DbContext`) que la representan. Útil para trabajar con bases de datos legadas.
 - *Code-First: (El enfoque que usaremos)* Tú defines primero tus clases C# (las Entidades POCO que representan tu modelo de dominio) y la clase `DbContext`. Luego, EF Core puede **generar automáticamente la base de datos** para que coincida con tu modelo, o **actualizar una base de datos existente** para reflejar los cambios en tus clases usando una característica llamada **Migrations**. Es ideal para nuevos proyectos donde tienes control total sobre el modelo.

(1.4) Modelando Entidades para EF Core (Code-First)

Las entidades son las clases POCO que definen la estructura de los datos que queremos persistir. EF Core utiliza una combinación de **convenciones** y **configuraciones explícitas** (Data Annotations o Fluent API) para entender cómo mapear estas clases a la base de datos.

- **Convenciones Comunes:** EF Core asume ciertas cosas por defecto para simplificar:
 - *Nombre de Clase -> Nombre de Tabla:* La clase `Producto` se mapea a una tabla llamada `Productos`.
 - *Propiedad Pública -> Columna:* La propiedad `public string Nombre { get; set; }` se mapea a una columna llamada `Nombre`.
 - *Clave Primaria (PK):* Una propiedad llamada `Id` o `NombreClaseId` (ej: `ProductoId`) se asume automáticamente como la PK.
 - *Tipos de Datos:* Mapea tipos C# a tipos SQL Server por defecto (ej: `string` -> `nvarchar(max)`, `int` -> `int`, `decimal` -> `decimal(18,2)`, `DateTime` -> `datetime2`, `bool` -> `bit`).
 - *Nulabilidad:* Propiedades de tipos por referencia (como `string`) o tipos Nullable (`int?`, `DateTime?`) se mapean a columnas que permiten `NULL`. Tipos por valor no nullable (`int`, `DateTime`, `bool`) se mapean a columnas `NOT NULL`.
- **Data Annotations (para Controlar el Mapeo):** A veces las convenciones no son suficientes o queremos ser más explícitos. Usamos atributos sobre las clases o propiedades (requieren `using System.ComponentModel.DataAnnotations;` y `using System.ComponentModel.DataAnnotations.Schema;`).

- `[Table("NombreEspecificoTabla")]`: Sobre la clase, para indicar un nombre de tabla diferente a la convención.
- `[Column("NombreColumnaDB", TypeName = "varchar(50)")]`: Sobre una propiedad, para especificar el nombre y/o el tipo SQL exacto de la columna.
- `[Key]`: Sobre una propiedad, para marcarla explícitamente como PK (útil si no se llama `Id` o `NombreClaseId`).
- `[Required]`: Sobre una propiedad, indica que la columna en la BD debe ser `NOT NULL`. (También usado por validación MVC).
- `[StringLength(100)]`: Indica longitud máxima para strings (mapea a `nvarchar(100)`).
- `[MaxLength(200)]`: Similar a `StringLength` para longitud máxima.
- `[ConcurrencyCheck]`: Para control de concurrencia optimista (más avanzado).
- `[Timestamp]`: Para columnas de versionado de fila (más avanzado).
- `[NotMapped]`: Sobre una propiedad, para decirle a EF Core que **ignore** esta propiedad, que no cree una columna para ella en la BD (útil para propiedades calculadas o temporales en la clase C#).
- `[ForeignKey("NombrePropiedadNavegacion")]`: Sobre la propiedad que representa la FK *explícitamente* (ej: `public int ClienteId { get; set; }`). Aunque a menudo no es necesario si sigues las convenciones de navegación.
- `[InverseProperty("NombrePropiedadNavegacionEnLaOtraClase")]`: Para desambiguar relaciones si hay múltiples relaciones entre las mismas dos tablas.
- **Modelando Relaciones (Propiedades de Navegación)**: EF Core detecta relaciones basándose en *propiedades de navegación* en tus clases.
 - **Relación Uno a Muchos (1:N)**: (Ej: Un *Cliente* tiene muchos *Pedidos*)

```
public class Cliente {
    public int Id { get; set; } // PK
    public string Nombre { get; set; }
    // Propiedad de navegación de colección: Un cliente tiene MUCHOS pedidos
    public virtual ICollection<Pedido> Pedidos { get; set; } = new List<Pedido>();
    // Inicializar la colección es buena práctica
    // 'virtual' habilita lazy loading (ver luego)
}

public class Pedido {
    public int Id { get; set; } // PK
    public DateTime Fecha { get; set; }

    // --- Clave Foránea (FK) ---
    public int ClienteId { get; set; } // Convención: NombreClasePadre + Id -> EF Core sabe que es FK

    // --- Propiedad de Navegación de Referencia ---
    // Un pedido pertenece a UN cliente
    public virtual Cliente Cliente { get; set; } // Apunta al objeto Cliente relacionado
}
```

EF Core infiere que **ClienteId** en **Pedido** es la FK que apunta a **Id** en **Cliente** debido a la combinación de la propiedad **ClienteId** y la propiedad de navegación **Cliente**. Creará la constraint FK en la BD automáticamente.

- **Relación Muchos a Muchos (N:N):** (Ej: Un *Producto* está en muchas *Categorías*, una *Categoría* tiene muchos *Productos*)
 - **Antes de EF Core 5:** Requería crear manualmente una clase y **DbSet** para la *tabla de unión* explícita (ej: **ProductoCategoria**).
 - **Desde EF Core 5:** Puedes modelarlo directamente con colecciones en ambos lados, y EF Core puede generar la tabla de unión *implícitamente* por convención (aunque a menudo sigue siendo útil definirla explícitamente si la relación tiene datos propios, como **FechaAsignacion**).

```
public class Producto {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public virtual ICollection<Categoria> Categorías { get; set; }
    = new List<Categoria>();
}
public class Categoria {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public virtual ICollection<Producto> Productos { get; set; } =
    new List<Producto>();
}
```

EF Core creará una tabla **ProductoCategoria** (o similar) con **ProductosId** y **CategoríasId** como FKs combinadas en una PK.

(1.5) El **DbContext**: Tu Puerta de Entrada a la Base de Datos

El **DbContext** es la clase más importante de EF Core para interactuar con la base de datos.

- *Rol Principal:*
 - Representa una **sesión** con la base de datos. Se crea una instancia (generalmente por petición web en ASP.NET Core) para realizar un conjunto de operaciones.
 - Actúa como una **Unidad de Trabajo (Unit of Work)**: Rastrea los cambios realizados en los objetos (entidades) durante su vida útil y persiste todos esos cambios a la base de datos de una sola vez cuando llamas a **SaveChanges()**.
 - Proporciona acceso a las **colecciones de entidades** a través de propiedades **DbSet<TEntity>**.
- *¿Cómo se crea?* Creas una clase que herede de **Microsoft.EntityFrameworkCore.DbContext**.

```
using Microsoft.EntityFrameworkCore; // Namespace principal

// Asumiendo que tenemos las clases Producto, Categoria, Cliente, Pedido...

public class MiAppDbContext : DbContext // Hereda de DbContext
{
    // Constructor necesario para pasarle las opciones de configuración (ej:
```

```

cadena de conexión)
    // Este constructor será usado por la Inyección de Dependencias
    public MiAppDbContext(DbContextOptions<MiAppDbContext> options) :
base(options)
    {
    }

    // Propiedades DbSet<T>: Una por cada entidad que quieras gestionar con
este contexto.
    // El nombre de la propiedad suele ser el plural del nombre de la
entidad.
    public DbSet<Producto> Productos { get; set; }
    public DbSet<Categoria> Categorias { get; set; }
    public DbSet<Cliente> Clientes { get; set; }
    public DbSet<Pedido> Pedidos { get; set; }
    // ... otros DbSets ...

    // (Opcional) Sobrescribir OnModelCreating para configuración avanzada
(Fluent API)
    // protected override void OnModelCreating(ModelBuilder modelBuilder) {
... }

    // (Opcional y MENOS recomendado en ASP.NET Core) Sobrescribir
OnConfiguring
    // protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder) {
    //     if (!optionsBuilder.IsConfigured) { // Solo si no se configuró
externamente (DI)
    //         optionsBuilder.UseSqlServer("Tu Connection String Aquí");
    //     }
    // }
}

```

- **DbSet<TEntity>**: Representa una colección de entidades (conceptualmente, una tabla) en el contexto. A través de estas propiedades (`_context.Productos`, `_context.Clientes`, etc.) realizarás las consultas (LINQ) y las operaciones CRUD.

Parte 2: Configuración y Gestión del Esquema (Migrations)

(2.1) Configurando el **DbContext** en ASP.NET Core (vía DI)

La forma estándar y recomendada de usar **DbContext** en ASP.NET Core es a través de la **Inyección de Dependencias (DI)**.

1. **Guardar la Cadena de Conexión:** En `appsettings.json` (o `appsettings.Development.json` para entorno de desarrollo).

```

{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost\\SQLEXPRESS;Database=MiAplicacionDB;Trusted_Connection=Tru

```

```
e;MultipleActiveResultSets=true;TrustServerCertificate=True"
    },
    "Logging": { /* ... */ }
    // ...
}
```

- Ajusta **Server** (puede ser `.` o `(localdb)\MSSQLLocalDB`) y **Database** (elige un nombre).
- **Trusted_Connection=True**: Usa autenticación de Windows. Si usas usuario/pass de SQL Server, sería `User ID=tu_usuario;Password=tu_clave;` y quitarías **Trusted_Connection**.
- **TrustServerCertificate=True**: A veces necesario para conexiones locales a SQL Express/Developer si no tienes un certificado válido configurado. ¡No usar en producción sin entender las implicaciones de seguridad!

2. Instalar Paquetes NuGet: Necesitas el proveedor de EF Core para tu base de datos y las herramientas.

- **PMC**: `Install-Package Microsoft.EntityFrameworkCore.SqlServer`
- **PMC**: `Install-Package Microsoft.EntityFrameworkCore.Tools`
- **.NET CLI**: `dotnet add package Microsoft.EntityFrameworkCore.SqlServer`
- **.NET CLI**: `dotnet add package Microsoft.EntityFrameworkCore.Tools` (y `dotnet tool install --global dotnet-ef`)

3. Registrar el Servicio DbContext en Program.cs:

```
// Al principio de Program.cs
using Microsoft.EntityFrameworkCore; // Namespace necesario
using MiPrimeraAppMVC.Data; // Namespace donde está tu DbContext (ajusta!)

var builder = WebApplication.CreateBuilder(args);

// --- Sección de Configuración de Servicios ---

// 1. Leer la cadena de conexión desde appsettings.json
var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");

// 2. Registrar MiAppDbContext como un servicio, configurándolo para usar
SQL Server
builder.Services.AddDbContext<MiAppDbContext>(options =>
    options.UseSqlServer(connectionString)); // ¡Así de simple!

// 3. Registrar otros servicios (MVC, etc.)
builder.Services.AddControllersWithViews();

// --- Fin de Configuración de Servicios ---

var app = builder.Build();

// --- Configuración del Pipeline de Middleware ---
// ... app.Use...

app.Run();
```

Ahora, el sistema de DI de ASP.NET Core sabe cómo crear instancias de `MiAppDbContext` y configurarlas con la cadena de conexión correcta.

4. Inyectar el `DbContext` donde se necesite (Controladores, Repositorios):

```
public class ProductosController : Controller
{
    private readonly MiAppDbContext _context; // Campo para guardar la
    instancia inyectada

    // Pedir la instancia en el constructor
    public ProductosController(MiAppDbContext context)
    {
        _context = context; // DI la provee automáticamente
    }

    public async Task<IActionResult> Index()
    {
        // Usar la instancia inyectada _context para consultar la BD
        var productos = await _context.Productos.Where(p =>
p.Activo).ToListAsync();
        // ...
        return View(/* ... */);
    }
}
```

- *Ciclo de Vida:* Por defecto, `AddDbContext` registra el contexto con un ciclo de vida "Scoped", lo que significa que se crea una nueva instancia de `MiAppDbContext` por cada petición HTTP que llega a la aplicación, y esa misma instancia se reutiliza si varios componentes la piden dentro de la misma petición. Al final de la petición, la instancia se "descarta" (*dispose*).

(2.2) EF Core Migrations: Manteniendo la Base de Datos Sincronizada

Una vez que tienes tu modelo C# (Entidades + `DbContext`) y la configuración, necesitas crear o actualizar la base de datos física para que coincida. **Migrations** es la herramienta de EF Core para esto en el enfoque Code-First.

- *¿Por qué usar Migrations?*
 - **Automatización:** Genera el SQL DDL (`CREATE TABLE`, `ALTER TABLE`, etc.) por ti.
 - **Control de Versiones:** Cada cambio en el esquema se representa como una migración (un archivo C# y un registro en la BD), lo que te permite versionar tu esquema junto con tu código fuente usando Git.
 - **Colaboración:** Facilita que varios desarrolladores trabajen en el mismo modelo y apliquen los cambios de esquema de forma consistente.
 - **Despliegue:** Permite aplicar los cambios de esquema de forma controlada en diferentes entornos (desarrollo, testing, producción).
- **Flujo de Trabajo Básico (usando Package Manager Console - PMC en Visual Studio):**
 - **Paso 0:** Asegúrate de tener el paquete `Microsoft.EntityFrameworkCore.Tools` instalado y selecciona el **proyecto que contiene tu `DbContext`** en el desplegable "Proyecto predeterminado" de la consola PMC.

- **Paso 1: Add-Migration NombreDeLaMigracion**

- Compara tu modelo C# actual con el último estado conocido por Migrations (guardado en un archivo "snapshot").
- Si detecta cambios, genera un nuevo archivo `.cs` en una carpeta `Migrations/`. Este archivo contiene:
 - Método `Up()`: Código C# que usa `MigrationBuilder` para describir los cambios DDL a aplicar (ej: `migrationBuilder.CreateTable(...)`, `migrationBuilder.AddColumn(...)`).
 - Método `Down()`: Código C# para *revertir* los cambios hechos por `Up()`. Permite deshacer una migración.
- Actualiza el archivo snapshot del modelo.
- **Nombres:** Usa nombres descriptivos para las migraciones, ej: `InitialCreate`, `AddProductosTable`, `AddEmailToClientes`, `RenamePrecioColumn`.

- **Paso 2: Update-Database**

- Este comando se conecta a la base de datos especificada en tu configuración (`appsettings.json` -> `Program.cs`).
- Comprueba la tabla especial `__EFMigrationsHistory` en esa base de datos para ver qué migraciones ya se han aplicado.
- Ejecuta el método `Up()` de todas las migraciones pendientes (las que se generaron pero no están en la tabla `__EFMigrationsHistory`), en el orden en que se crearon. EF Core traduce el código C# del `MigrationBuilder` a SQL DDL y lo ejecuta.
- Si la base de datos no existe, **Update-Database** la **creará automáticamente** antes de aplicar las migraciones.
- Registra las migraciones aplicadas en la tabla `__EFMigrationsHistory`.

- **Ciclo de Desarrollo:**

1. Modificas tus clases Entidad o `DbContext` en C#.
2. Ejecutas `Add-Migration NombreDelCambio` en PMC. Revisas la migración generada.
3. Ejecutas `Update-Database` en PMC para aplicar los cambios a tu base de datos de desarrollo.
4. Verificas los cambios en la BD usando SSMS/Azure Data Studio.
5. Commiteas tu código C# **Y** la carpeta `Migrations` a Git.

- **Otros Comandos Útiles (PMC):**

- `Remove-Migration`: Borra la última migración *generada*. **¡Solo usar si NO has ejecutado Update-Database después de Add-Migration!** Si ya aplicaste la migración, necesitas revertirla primero (`Update-Database NombreMigracionAnterior`) o manejarlo manualmente (con cuidado).
- `Update-Database NombreMigracionDestino`: Aplica o revierte migraciones hasta llegar al estado de `NombreMigracionDestino`. Usar `Update-Database 0` revierte todas las migraciones.
- `Script-Migration [-From MigracionInicial] [-To MigracionFinal]`: Genera un script SQL con los cambios DDL, en lugar de aplicarlos directamente.

Parte 3: Consultando Datos - LINQ to Entities

Ahora que tenemos la BD sincronizada con el modelo, podemos consultar los datos usando **LINQ (Language-Integrated Query)** sobre nuestro `DbContext`.

(3.1) LINQ: Consultas Integradas en C#

LINQ nos permite escribir consultas de forma declarativa y fuertemente tipada directamente en C#.

- ¿Por qué usar LINQ en lugar de escribir SQL como strings?
 - **Seguridad de Tipos y Compilación:** Los errores de sintaxis o nombres incorrectos de tablas/columnas (propiedades en C#) se detectan al compilar, no en ejecución.
 - **IntelliSense:** Autocompletado y ayuda del editor al escribir consultas.
 - **Depuración:** Puedes poner breakpoints y depurar tus consultas LINQ.
 - **Composición:** Fácil construir consultas complejas encadenando operaciones LINQ.
 - **Abstracción:** Menos dependencia del dialecto SQL específico del motor de BD.
- **LINQ to Entities:** Cuando aplicamos LINQ sobre un `DbSet<TEntity>` de EF Core (`_context.Productos`), EF Core actúa como un **traductor**. Convierte nuestra expresión LINQ (escrita en C#) en una consulta SQL optimizada que se ejecuta en el servidor de base de datos.

(3.2) `IQueryable<T>` y Ejecución Diferida (Deferred Execution)

Cuando escribes `var consulta = _context.Productos.Where(p => p.Activo);`, **no** se ejecuta ninguna consulta SQL todavía. `consulta` es de tipo `IQueryable<Producto>`, que representa la *intención* de la consulta, no los resultados. Esto permite **encadenar** más operadores LINQ (`.OrderBy()`, `.Select()`, etc.) para construir la consulta completa.

La consulta SQL **solo se genera y ejecuta** contra la base de datos cuando *materializas* los resultados, típicamente llamando a métodos como:

- `.ToList()` / `.ToListAsync()`
- `.ToArray()` / `.ToArrayAsync()`
- `.FirstOrDefault()` / `.FirstOrDefaultAsync()` (y similares: `First`, `Single`, etc.)
- `.Count()` / `.CountAsync()`
- `.Any()` / `.AnyAsync()`
- Iterando con `foreach` (esto también fuerza la ejecución).

(3.3) Operadores LINQ Comunes (Sintaxis de Métodos)

(Asumiendo `_context` es la instancia inyectada de `MiAppDbContext` y las acciones son `async Task<IActionResult>`)

- **Obtener Todos:**

```
List<Producto> todos = await _context.Productos.ToListAsync();
```

- **Filtrar (`Where`):**

```
var baratosActivos = await _context.Productos
    .Where(p => p.Precio < 1000 && p.Activo)
    .ToListAsync();

string buscar = "García";
var clientesGarcia = await _context.Clientes
    .Where(c => c.Apellido == buscar)
    .ToListAsync();
```

- **Ordenar (OrderBy, OrderByDescending, ThenBy, ThenByDescending):**

```
var productosPorNombre = await _context.Productos
    .OrderBy(p => p.Nombre)
    .ThenByDescending(p => p.Precio) //
Segundo criterio
    .ToListAsync();
```

- **Proyección (Select):** Seleccionar solo algunas columnas o transformar el resultado.

```
// A tipo anónimo
var proyeccionSimple = await _context.Productos
    .Select(p => new { IdProd = p.Id, NombreProd
= p.Nombre })
    .ToListAsync();

// A ViewModel (muy común)
var productosVM = await _context.Productos
    .Where(p => p.Activo)
    .Select(p => new ProductoListViewModel {
        Id = p.Id,
        NombreCompleto = p.Nombre + " (Activo)",
        PrecioVenta = p.Precio * 1.21m // Calcular
algo
    })
    .ToListAsync();
```

- **Obtener Uno Solo:**

```
int idBuscado = 5;
Producto producto = await _context.Productos.FirstOrDefaultAsync(p => p.Id
== idBuscado);
// producto será null si no se encuentra el ID 5

Producto otroProd = await _context.Productos.FindAsync(idBuscado); //
Optimizado para PK
```

- **Paginación (Skip, Take):** Para mostrar resultados en páginas.

```
int pagina = 2;
int tamañoPagina = 10;
var productosPagina2 = await _context.Productos
    .OrderBy(p => p.Nombre)
    .Skip((pagina - 1) * tamañoPagina) //
Salta los de páginas anteriores
    .Take(tamañoPagina) // Toma solo los de
```

esta página

.ToListAsync();

(3.4) Cargando Datos Relacionados (**Include**, **ThenInclude**)

- **El Problema N+1:** Si cargas entidades (ej: Pedidos) y luego, al iterar sobre ellas, accedes a una propiedad de navegación (ej: `pedido.Cliente.Nombre`), EF Core podría ejecutar una consulta adicional por *cada* pedido para traer su cliente. ¡Muy ineficiente!
- **Solución: Eager Loading (Carga Ansiosa) con **Include**:** Le dices a EF Core que traiga los datos relacionados en la consulta inicial usando un **JOIN** en SQL.

```
var pedidos = await _context.Pedidos
    .Include(p => p.Cliente) // Incluye la info del
    Cliente
    .Where(p => p.Fecha > DateTime.Now.AddMonths(-1))
    .ToListAsync();

// Carga multinivel y colecciones
var clienteConPedidosYProductos = await _context.Clientes
    .Include(c => c.Pedidos) // Incluye la colección de Pedidos del
    cliente
    .ThenInclude(p => p.Items) // Dentro de cada Pedido, incluye sus
    Items
    .ThenInclude(i => i.Producto) // Dentro de cada Item,
    incluye el Producto
    .FirstOrDefaultAsync(c => c.Id == clienteId);
    // Ahora puedes navegar c.Pedidos[0].Items[0].Producto.Nombre sin
    más queries
```

- Usa **Include** para las relaciones que *sabes* que vas a necesitar inmediatamente.

(3.5) Operaciones Asíncronas (**async/await**)

- **¡Fundamental en ASP.NET Core!** Todas las operaciones de EF Core que involucran I/O con la base de datos (queries, **SaveChanges**) tienen versiones asíncronas (terminan en **Async**).
- *¿Por qué?* Para no bloquear el hilo del servidor mientras espera la respuesta de la BD. Esto permite que el servidor maneje muchas más peticiones concurrentes, mejorando la escalabilidad.
- *¿Cómo?*
 - Marca tu método de acción con **async Task<IActionResult>**.
 - Usa **await** antes de llamar a los métodos **Async** de EF Core (ej: `await _context.ToListAsync()`, `await _context.SaveChangesAsync()`).

Parte 4: Modificando Datos (CRUD) y Patrones de Diseño (Repositorio)

(4.1) Operaciones CRUD con EF Core

EF Core simplifica enormemente las operaciones de Crear, Leer (ya visto), Actualizar y Eliminar. La clave es entender el **seguimiento de cambios (change tracking)** y el método **SaveChanges**.

- **Change Tracking:** Cuando recuperas una entidad de la BD con EF Core (ej: usando `FindAsync` o `FirstOrDefaultAsync`), el `DbContext` mantiene una referencia a ese objeto y "vigila" si sus propiedades cambian.
- **Estados de Entidad:** EF Core asigna un estado a cada entidad que rastrea:
 - *Detached*: El contexto no sabe nada de esta entidad.
 - *Unchanged*: El contexto la rastrea, pero sus valores no han cambiado desde que se leyó de la BD.
 - *Added*: La entidad es nueva y se ha añadido al contexto (`_context.Add(nuevaEntidad)`). Se insertará en `SaveChanges`.
 - *Modified*: El contexto la rastrea y detectó que al menos una de sus propiedades ha cambiado. Se actualizará en `SaveChanges`.
 - *Deleted*: La entidad ha sido marcada para eliminación (`_context.Remove(entidad)`). Se borrará en `SaveChanges`.
- **SaveChanges() / SaveChangesAsync():** Este es el método que **persiste** todos los cambios acumulados (entidades *Added*, *Modified*, *Deleted*) en la base de datos. Revisa el estado de todas las entidades rastreadas, genera el SQL apropiado (INSERT, UPDATE, DELETE) y lo ejecuta, usualmente dentro de una **transacción** para asegurar que todas las operaciones se completen con éxito o ninguna lo haga (atomicidad). Devuelve el número de filas afectadas.
- **Create (INSERT):**

```
[HttpPost]
public async Task<IActionResult> Create(ProductoViewModel vm)
{
    if (ModelState.IsValid) {
        Producto nuevoProducto = new Producto { /* mapear desde vm */ };
        _context.Productos.Add(nuevoProducto); // Estado: Added
        await _context.SaveChangesAsync(); // Ejecuta INSERT, actualiza
        nuevoProducto.Id
        return RedirectToAction("Index");
    }
    return View(vm);
}
```

- **Update (UPDATE - Enfoque Conectado):**

```
[HttpPost]
public async Task<IActionResult> Edit(int id, ProductoViewModel vm)
{
    if (id != vm.Id) return BadRequest(); // Validar ID

    if (ModelState.IsValid) {
        Producto productoFromDb = await _context.Productos.FindAsync(id); //
        Estado: Unchanged (o Detached si no lo encuentra)
        if (productoFromDb == null) return NotFound();

        // Modificar propiedades del objeto rastreado
    }
}
```

```

        productoFromDb.Nombre = vm.Nombre;
        productoFromDb.Precio = vm.Precio;
        // ... mapear otras ...
        // EF Core detecta los cambios y cambia el estado a Modified

        try {
            await _context.SaveChangesAsync(); // Ejecuta UPDATE solo para
Nombre y Precio
        } catch (DbUpdateConcurrencyException) {
            // Manejar posible error si otro usuario modificó/borró el
registro mientras tanto
            throw;
        }
        return RedirectToAction("Index");
    }
    return View(vm);
}

```

- **Delete (DELETE):**

```

[HttpPost, ActionName("Delete")] // Para evitar colisión con GET/Delete/id
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    Producto productoAEliminar = await _context.Productos.FindAsync(id);
    if (productoAEliminar != null) {
        _context.Productos.Remove(productoAEliminar); // Estado: Deleted
        await _context.SaveChangesAsync(); // Ejecuta DELETE
    }
    return RedirectToAction("Index");
}

```

(4.2) Introducción al Patrón Repositorio

- *El Problema:* Aunque EF Core simplifica mucho, poner lógica de `_context.Productos.Where(...).Include(...).ToListAsync()` o `_context.Add/Remove/SaveChanges` directamente en los Controladores puede llevar a:
 - **Controladores "gordos":** Mucha lógica que no es estrictamente de manejo de petición/respuesta.
 - **Acoplamiento:** El Controlador depende directamente de `MiAppDbContext` y de los detalles de EF Core.
 - **Dificultad para Probar:** Probar unitariamente el Controlador requiere un `DbContext` real o un setup complejo para simularlo.
 - **Repetición de Código:** Consultas LINQ comunes pueden repetirse en varias acciones.
- *La Solución (Patrón Repositorio):* Crear una **capa de abstracción** dedicada exclusivamente al acceso a datos.
 - **Componentes:**

1. **Interfaz del Repositorio (*IProductoRepository*):** Define un **contrato** claro de las operaciones de datos necesarias para una entidad (*Producto* en este caso), *sin* especificar cómo se implementan. Métodos como *GetByIdAsync(int id)*, *GetAllAsync()*, *AddAsync(Producto p)*, *Update(Producto p)*, *Remove(Producto p)*.
 2. **Implementación Concreta (*ProductoRepository*):** Una clase que **implementa** la interfaz *IProductoRepository*. Esta clase **sí** depende del *DbContext* y contiene la lógica de EF Core (LINQ, Add, Remove, etc.) para cumplir con el contrato de la interfaz.
 3. **Inyección de Dependencias:**
 - Registrar en *Program.cs*:
`builder.Services.AddScoped<IProductoRepository, ProductoRepository>();` (Le dice a DI que cuando alguien pida *IProductoRepository*, debe crear una instancia de *ProductoRepository*).
 - Inyectar la **Interfaz** en el Constructor del Controlador: `public ProductosController(IProductoRepository repo) { _repo = repo; }.`
- **Ventajas:**
 - **Desacoplamiento:** El Controlador solo conoce la *interfaz* del repositorio, no los detalles de EF Core. Si cambias de EF Core a Dapper, solo modificas *ProductoRepository*, el Controlador no se entera.
 - **Testabilidad:** Puedes crear fácilmente una implementación "falsa" o "mock" de *IProductoRepository* que devuelva datos de prueba en memoria, y usarla para probar unitariamente el Controlador sin base de datos.
 - **Centralización:** La lógica de acceso a datos para *Producto* está en un solo lugar (*ProductoRepository*).
 - **Consideración con EF Core:** Como *DbContext* y *DbSet* ya ofrecen mucha abstracción (son en sí mismos una forma de Unit of Work y Repository genérico), crear repositorios específicos *adicionales* puede parecer redundante para operaciones CRUD simples. Sin embargo, **agregan valor significativo** cuando:
 - Necesitas centralizar consultas LINQ complejas o reutilizables.
 - Quieres una capa clara para aplicar lógica específica de acceso a datos.
 - La testabilidad unitaria de los controladores es una alta prioridad.
 - Consideras la posibilidad futura de cambiar la tecnología de persistencia.
 - **Patrón Unit of Work (Mención):** A menudo se usa junto con el Repositorio. La idea es tener una clase (*UnitOfWork*) que contenga referencias a varios repositorios (*IProductoRepository*, *IClienteRepository*, etc.) y un único método *SaveChangesAsync()*. Esto asegura que los cambios hechos a través de diferentes repositorios dentro de una misma operación de negocio se guarden juntos en una sola transacción. El *DbContext* de EF Core ya actúa como una Unit of Work, por lo que a veces un Unit of Work explícito no es estrictamente necesario si solo usas un *DbContext*.

Conclusión

Entity Framework Core es una herramienta poderosa que actúa como puente entre nuestro mundo orientado a objetos en C# y el mundo relacional de SQL Server. Usando el enfoque Code-First, podemos definir nuestro modelo de datos en C# y mantener la base de datos sincronizada con Migrations. LINQ to Entities nos ofrece una forma segura y expresiva de consultar los datos, mientras que los métodos del *DbContext* simplifican las operaciones CRUD. Configurar *DbContext* con Inyección de Dependencias es esencial en ASP.NET Core. Finalmente, patrones como el Repositorio (y Unit of Work) nos ayudan a escribir código más limpio, desacoplado y testeable, separando las responsabilidades del acceso a datos de la lógica de la aplicación.

Dominar EF Core es fundamental para desarrollar aplicaciones .NET que interactúen con bases de datos de manera eficiente y mantenible.

Síntesis de una oración: Este material detalla cómo usar Entity Framework Core en ASP.NET Core con el enfoque Code-First para mapear objetos C# a bases de datos SQL Server, gestionar el esquema con Migrations, consultar datos con LINQ to Entities, realizar operaciones CRUD y aplicar patrones como el Repositorio mediante Inyección de Dependencias.

Síntesis de 5 puntos importantes:

1. **EF Core** es un **ORM** que traduce entre objetos C# y tablas de BD, facilitando el acceso a datos; **Code-First** permite definir la BD desde clases C#.
2. El **DbContext** representa la sesión con la BD y expone **DbSet<T>** para cada tabla; se configura vía **DI** en *Program.cs* (**AddDbContext**, **UseSqlServer**).
3. **Migrations** (*Add-Migration*, *Update-Database*) son esenciales en Code-First para crear y actualizar el esquema de la BD basado en cambios en las clases Entidad.
4. **LINQ to Entities** permite escribir consultas seguras y tipadas en C# (**Where**, **Select**, **OrderBy**, etc.) que EF Core traduce a SQL; **Include** se usa para cargar datos relacionados (evitar N+1).
5. Las operaciones **CRUD** se realizan con métodos del **DbContext** (**AddAsync**, **FindAsync**, **Remove**, **Update**/ChangeTracking) y se persisten con **SaveChangesAsync**; el **Patrón Repositorio** ayuda a desacoplar esta lógica de los Controladores.

55286